

FSDP：完全分片数据并行

原始问题：详细讲解大模型中的 FSDP 技术，并解释和其他并行策略有什么区别，最后总结成表格列出异同和适合场景。

提问：2026/07/26 15:09 更新：2026/07/27 01:53 重要度：5/5

一句话结论

FSDP (Fully Sharded Data Parallel, 完全分片数据并行) 的本质是：在数据并行的语义下，把每张卡上“完整复制一份模型训练状态”改成“每张卡只存 $1/N$ 份参数、梯度和优化器状态”；在真正要用到某一层时，才临时用一次 AllGather 通信把这一层的完整参数拼回来，算完立刻丢掉。

它用通信换显存：把 DDP 里冗余复制的内存开销摊薄到 N 张卡上，从而能训练远超单卡显存的大模型。它是 ZeRO (Zero Redundancy Optimizer) 思想在 PyTorch 中的原生实现。

系统位置：它解决/不解决什么

解决的核心问题：单卡装不下完整训练状态。一个 7.5B 模型混合精度+Adam 需 $16 \times 7.5e9 = 120\text{GB}$ ，单张 80GB H100 都装不下，而 DDP 每张卡都存这 120GB，纯属冗余。

不解决：单层本身太大装不下（需 TP 切算子）；层数太多单卡放不下计算图（需 PP）。它减少的是显存不是计算量——每卡 FLOP 一点没少。

上下游：属于数据并行维度的增强版，可与 TP/PP/EP 正交组合成 3D/4D 并行，替代的是 DDP。

直觉：共享工具间的工厂

DDP：N 个工人每人一整套完整工具（全部参数/梯度/优化器状态），N 套占 N 倍仓库空间，极度浪费。

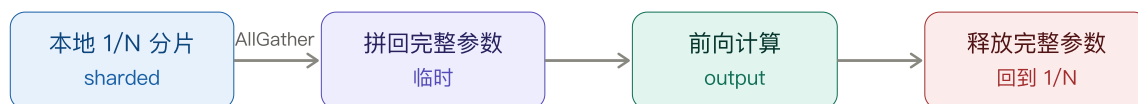
FSDP：N 个工人共用中央工具库，每人只保管 $1/N$ 工具。轮到用某把扳手（某一层）时，保管它的工人广播复印给所有人（AllGather），大家一起用，用完立刻销毁复印件，只有原保管人留正本。

关键代价：每次用工具都要现场“复印+销毁”（通信+临时显存峰值）。类比失效点：现实中复印 (AllGather) 和干活 (计算) 可重叠——用当前扳手时提前复印下一把 (prefetch)。

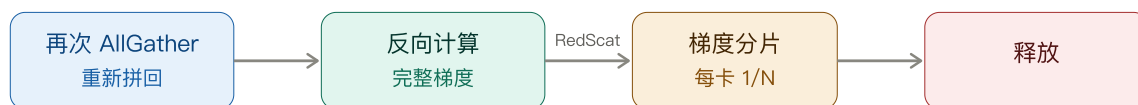
关键图：单层参数生命周期时序

FSDP 一层生命周期：AllGather 拼回 → 计算 → 立即释放

前向传播



反向传播



三个关键通信原语

- AllGather：把散在 N 卡上的参数分片拼成完整层参数（前向、反向各一次）
- ReduceScatter：对完整梯度求和规约，同时按分片切回，每卡只留 1/N 梯度
- 对比 DDP：DDP 只在反向做一次 AllReduce = AllGather + ReduceScatter 的合体

核心权衡：FSDP 通信量约为 DDP 的 1.5 倍，换来每卡显存降到约 1/N

前向多出的那次 AllGather 是 FSDP 比 DDP 多的主要通信开销来源

优化器状态全程分片、永不拼回，是省显存最狠的一块

机制：一个训练 step 里做什么

调度单位是 FSDP unit（通常一层）。

前向：1) 本地只存 1/N 参数分片；2) AllGather 拼回完整参数（临时）；3) 前向计算；4) 立即释放完整参数。

反向：5) 再次 AllGather（因为完整参数已被释放）；6) 反向计算得完整梯度；7) ReduceScatter 对梯度求和并按分片切回，每卡只留 1/N；8) 释放完整参数。

优化器更新：每卡用自己的 1/N 梯度+1/N fp32 参数副本+1/N 优化器状态，独立更新 1/N 参数。优化器状态全程分片、永不拼回，这是省显存最狠的一块。

与 DDP 通信对比：DDP 反向做 1 次 AllReduce；FSDP 做 前向AllGather + 反向AllGather + 反向 ReduceScatter，通信量约 DDP 的 1.5 倍。

公式与量级

每参数训练状态 ≈ 16 Byte
= fp16参数(2) + fp16梯度(2) + fp32副本(4) + 动量m(4) + 方差v(4)

每卡常驻训练状态 (不含激活):

DDP : $M = 16 \times \Psi$ (完整复制, 与 N 无关)
FSDP ZeRO-3: $M \approx 16 \times \Psi / N$ (全部摊薄到 N 卡)

数值算例 ($\Psi=7.5B, N=64$):

DDP : $16 \times 7.5e9 = 120$ GB / 卡 $\rightarrow$ 放不下
FSDP : $(16 \times 7.5e9) / 64 \approx 1.875$ GB / 卡 $\rightarrow$ 轻松放下

通信量: FSDP / DDP $\approx 3P / 2P = 1.5$ (多了前向 AllGather)

重要: 以上只算参数相关状态, 未含激活。FSDP 默认不分片激活, 大 batch/长序列时激活可能才是 OOM 主因。

最小 Sample: PyTorch FSDP

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import ShardingStrategy, BackwardPrefetch
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
import functools, torch.nn as nn

# 关键点1: 按 transformer block 切分 FSDP unit
policy = functools.partial(transformer_auto_wrap_policy,
                           transformer_layer_cls={nn.TransformerEncoderLayer})

model = FSDP(
    model,
    auto_wrap_policy=policy,
    # 关键点2: FULL_SHARD=ZeRO-3 (全分片); SHARD_GRAD_OP=ZeRO-2
    sharding_strategy=ShardingStrategy.FULL_SHARD,
    # 关键点3: 反向预取, 让 AllGather 与 反向计算重叠
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
)
# 训练循环与 DDP 几乎一样, 分片/拼回/释放由 FSDP 内部完成
for batch in loader:
    loss = model(batch).loss
    loss.backward() # 内部自动 AllGather + ReduceScatter
    optimizer.step() # 每卡只更新自己的 1/N 参数
```

应用场景 (何时用 FSDP)

- 模型能装进单卡只想加速 $\rightarrow$ 用 DDP (FSDP 额外通信不划算)
- 训练状态超单卡但单层能装下 $\rightarrow$ 首选 FSDP
- 单层太大装不下 $\rightarrow$ FSDP + TP
- 层数极多想提利用率 $\rightarrow$ FSDP + PP
- MoE 大模型 $\rightarrow$ FSDP + EP

- 显存极紧 → FSDP + CPU offload

常见误区

- 1) 误以为 FSDP 减少计算量：错，只减显存，每卡 FLOP 与 DDP 一样。
- 2) 误以为用了 FSDP 显存就一定够：错，默认不分片激活，大 batch/长序列仍可能 OOM。
- 3) 把 FSDP 和 TP 混为一谈：FSDP 计算某层时拼回完整参数再算（层计算完整）；TP 是这一层计算本身被切开多卡协同。切分维度不同，可叠加。
- 4) 以为 ZeRO 和 FSDP 是两个东西：ZeRO 是 DeepSpeed 的算法思想，FSDP 是 PyTorch 原生实现，FULL_SHARD≈ZeRO-3。
- 5) 切分粒度随意设：太细通信次数暴涨，太粗临时参数撑爆显存峰值，需按 block 粒度并 profiler 调。